Gradient Descent — Lecture Notes

CS 3600 — Intro to AI | Lecture 8: Optimization

1. Intuition

Training a model means finding the parameters that make its predictions less wrong. A loss function turns that wrongness into a single score, and lower is better.

Gradient descent minimizes a loss function by moving opposite the slope.

Picture the loss landscape as a valley. Standing anywhere on its surface, you can feel which way the ground tilts. The slope at your feet points uphill, so you step the other way and repeat. You never need a map of the whole valley, only the local slope.

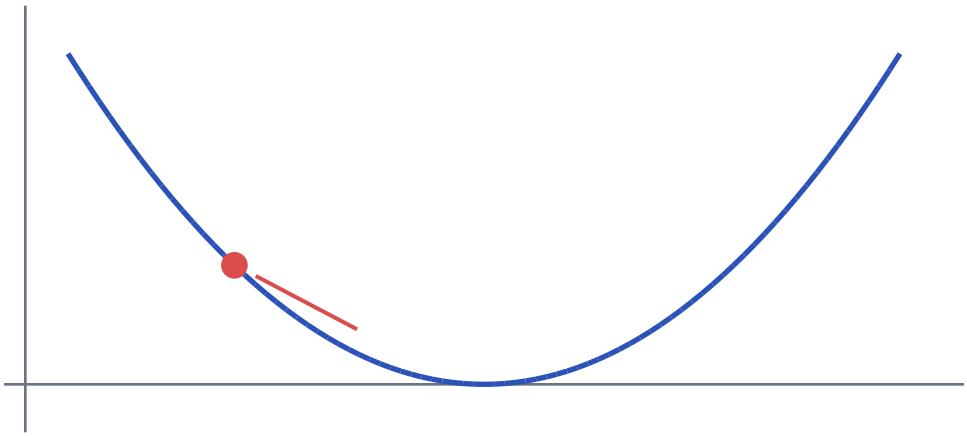

Figure 1. A one-parameter loss curve. The minimum is where the slope is zero.

2. The Algorithm

Start from an initial guess for the weight w. At each iteration, compute the gradient of the loss at the current weight, then update the weight:

```
w_next = w_current - alpha * gradient
```

The gradient points in the direction of steepest increase, so subtracting it moves the weight downhill. Here alpha is the learning rate. It scales every update, which sets how far each step travels.

Choosing alpha is a tradeoff between slow convergence and overshooting:

- Too small: updates are tiny, so training converges slowly.
- Too large: updates jump past the minimum and can overshoot it, or even diverge.

Stop when the gradient is close to zero or when the loss stops improving.

3. Worked Example

Take a loss whose minimum sits at w = 3:

```
L(w) = (w - 3)^2
```

```
gradient = 2(w - 3)
```

```
alpha = 0.4, starting at w = 0
```

The first update gives w_next = 0 - 0.4 * (-6) = 2.4. Repeating the rule produces:

Iteration	w	gradient	L(w)
0	0	-6	9
1	2.4	-1.2	0.36
2	2.88	-0.24	0.0144
3	2.976	-0.048	0.000576

The loss falls from 9 to 0.36, then 0.0144, then 0.000576. Each update shrinks the distance to the minimum by the same factor, so the steps get smaller as the slope flattens.